Table of Contents

**Table of Contents** **1**
**1. Project Overview** **1**
**2. Class Diagrams** **2**
**2.1 Domain Models** **2**
**2.2 Design Patterns** **3**
**3. Database Schema** **9**
**3.1 Entity-Relationship Diagram (PlantUML)** **9**
**3.2 Table Descriptions** **9**
Users 9
Cars 10
Bookings 10
Messages 11
Notifications 12
Reviews 12
**4. User Interface Screenshots** **13**
4.1 Authentication Pages 13
4.2 Dashboard & Navigation 15
4.3 Car Management 15
4.4 Booking System 17
4.5 Watch & Notifications 18
4.6 Messaging 19
4.7 Reviews & Profile 19

1. Project Overview

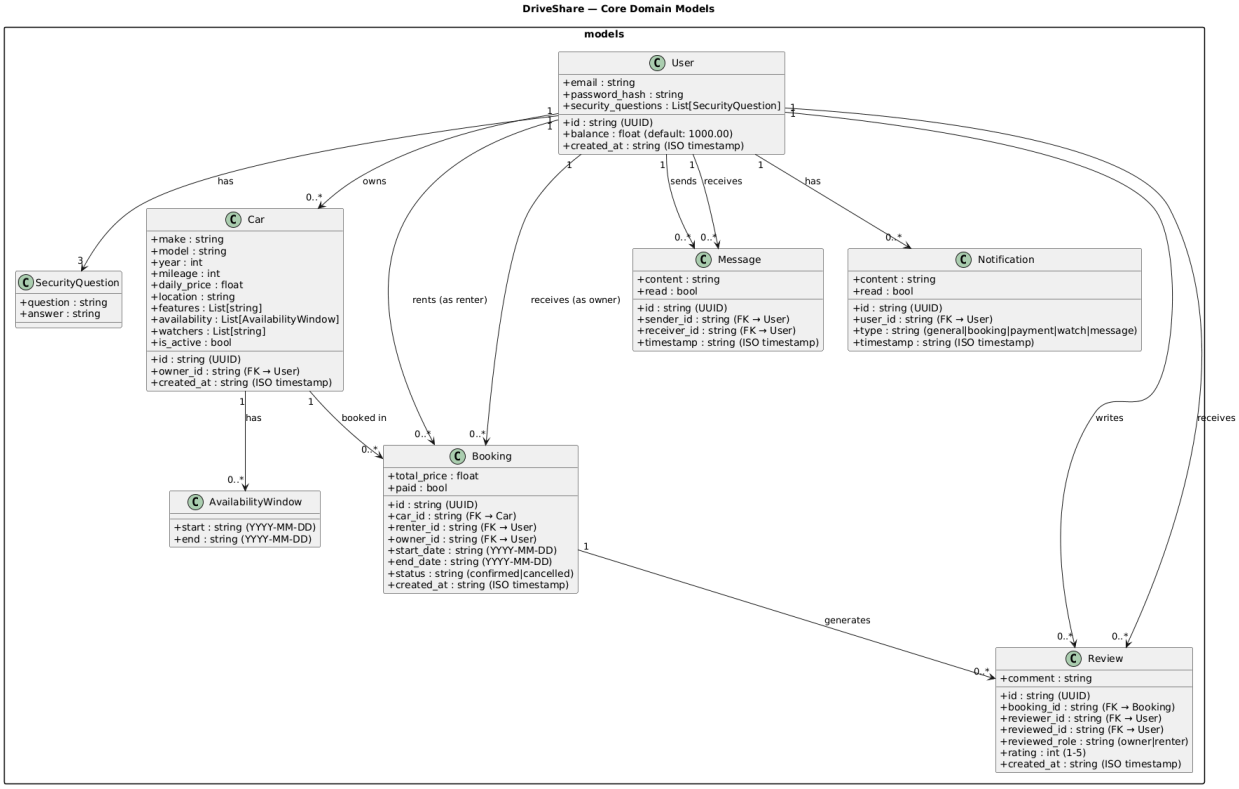
**DriveShare** is a peer-to-peer car rental platform inspired by Turo.com. The application allows car owners to list their vehicles for rent and renters to search, book, and pay for rentals. The system implements six design patterns and features a React frontend with a Python/FastAPI backend using SQLite for persistence.

**Key Features**

- User registration and authentication with SHA-256 password hashing
- Security questions for password recovery (Chain of Responsibility pattern)
- Car listing management with Builder pattern
- Search and booking with overlap prevention
- Watch system for price drop notifications (Observer pattern)
- In-app messaging between users
- Simulated payment processing (Proxy pattern)
- Session management (Singleton pattern)
- UI component coordination (Mediator pattern)

2. Class Diagrams

2.1 Domain Models

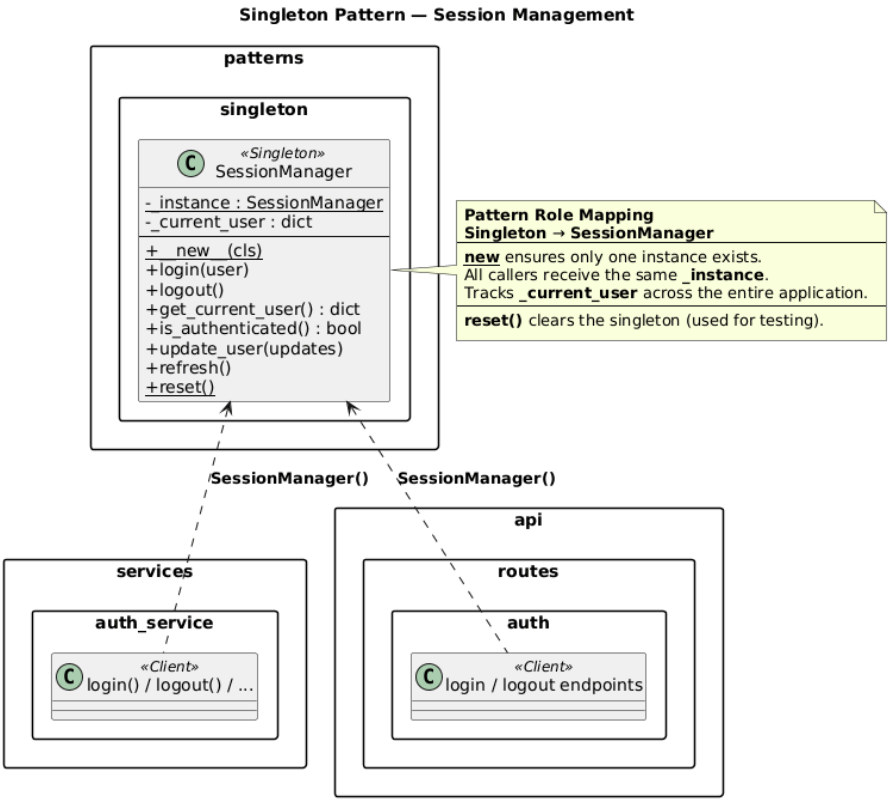



**Description:** The Core Domain Model diagram illustrates the main entities in the DriveShare system and their relationships:

- **User:** Central entity representing both car owners and renters. Each user has 3 security questions for password recovery and a starting balance of $1,000.
- **Car:** Represents a vehicle listing with details like make, model, year, pricing, and availability windows. Linked to its owner and can have multiple watchers (users monitoring price changes).
- **Booking:** Captures a rental transaction between a renter and owner for a specific car and date range. Tracks payment status.
- **Message:** Enables direct communication between users (typically renter and owner).
- **Notification:** System-generated alerts for bookings, payments, price drops, and messages.
- **Review:** Post-rental feedback where renters review owners and vice versa.

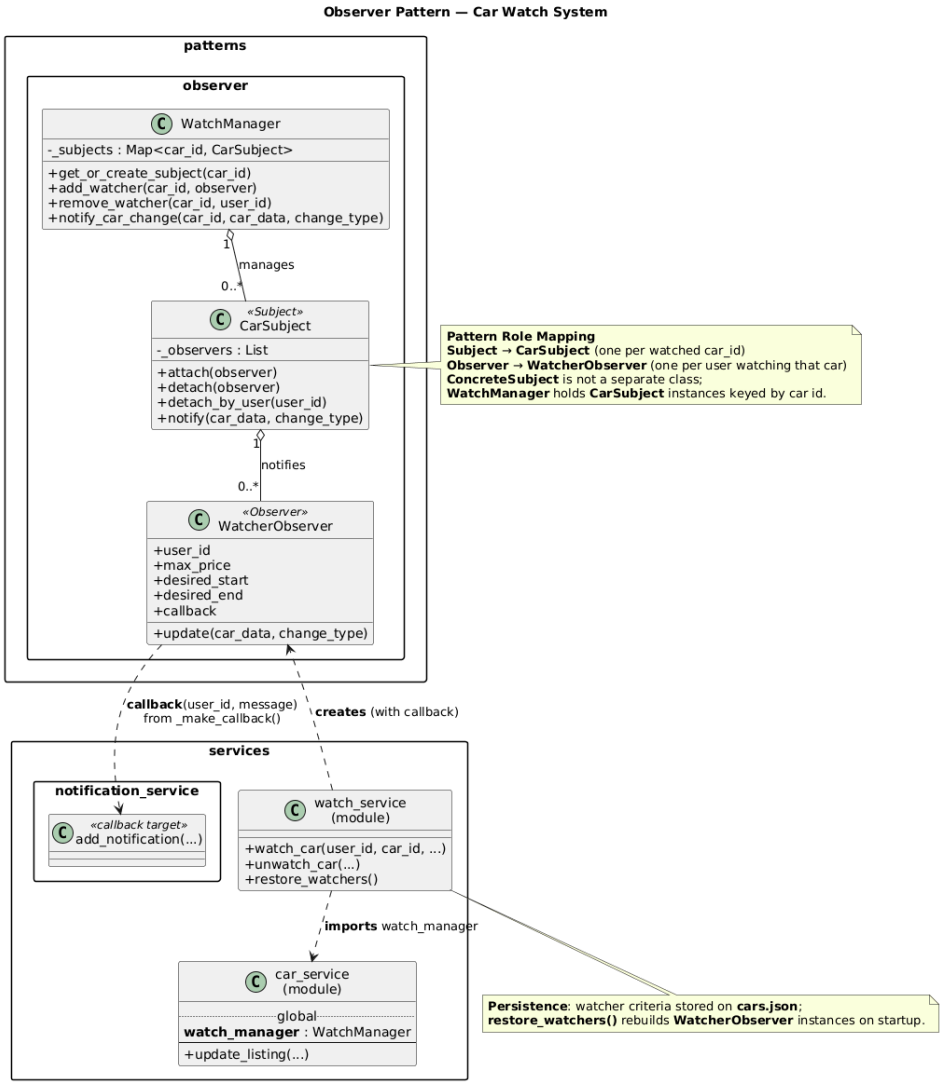
## 2.2 Design Patterns

### 2.2.1 Singleton Pattern: Session Management



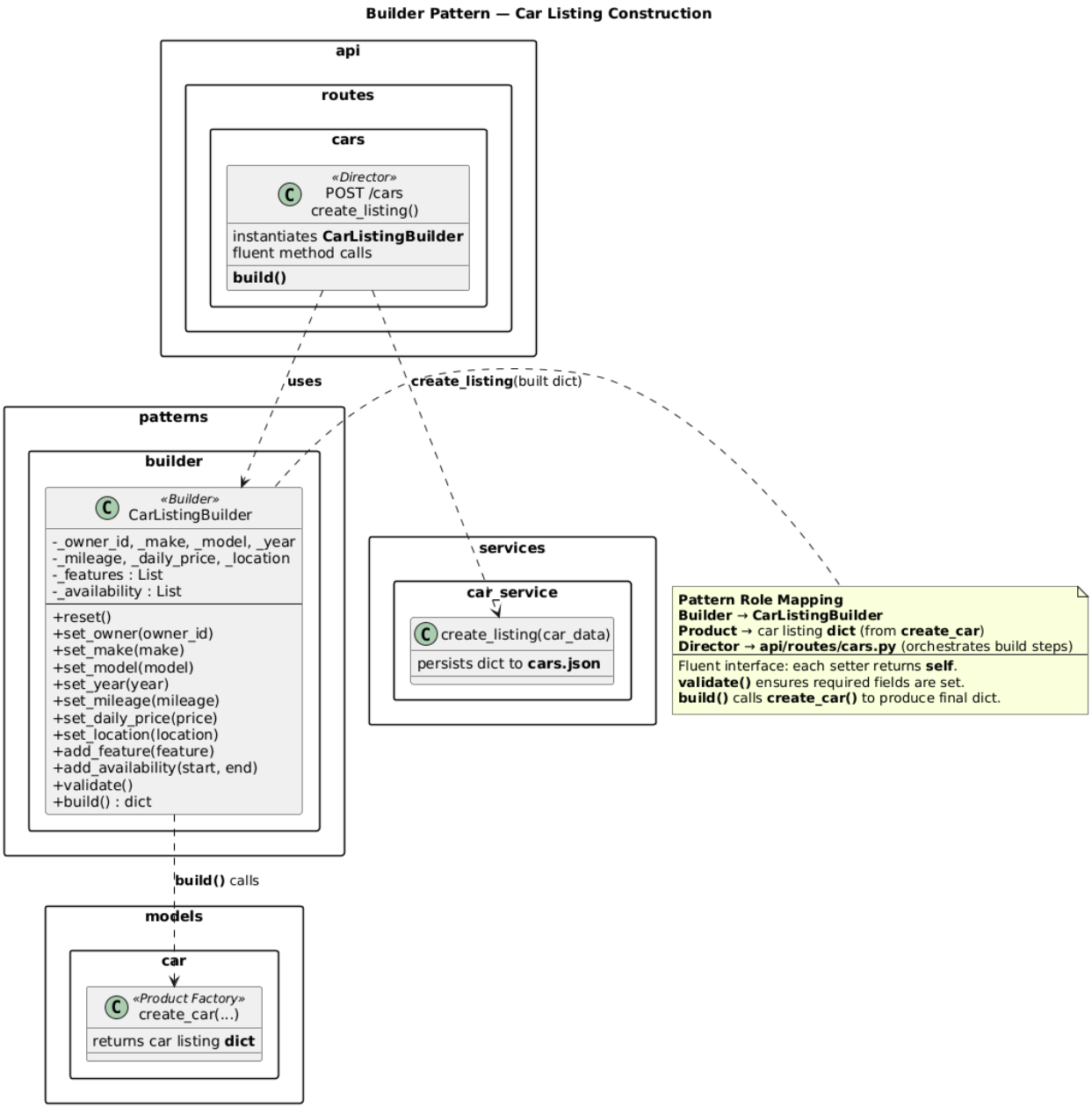

**Description:** The Singleton pattern ensures only one SessionManager instance exists throughout the application lifecycle. This is critical for maintaining a consistent user session state across all components. The **__new__** method intercepts object creation and returns the already existing instance if one exists. Key operations include **login()** to set the current user, **logout()** to clear the session, and **refresh()** to reload user data from storage (useful after balance changes from payments).

### 2.2.2 Observer Pattern: Car Watch System



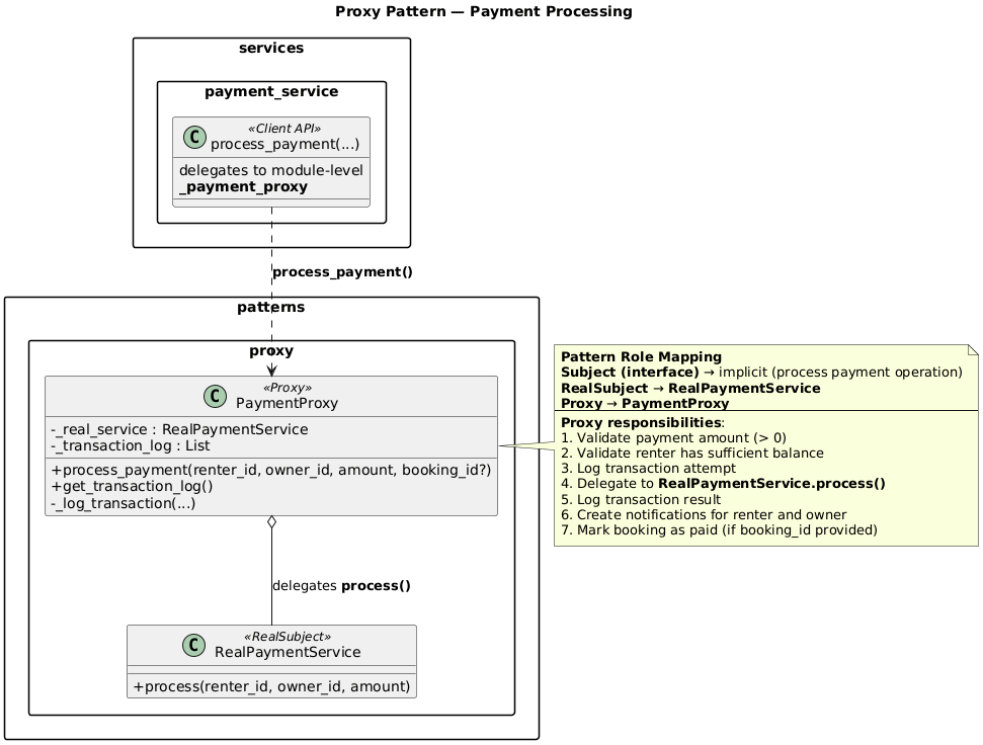

**Description:** The Observer pattern implements the "Watch" feature where renters can subscribe to car listings and receive notifications when prices drop or availability changes. Each CarSubject maintains a list of WatcherObserver instances. When car_service.update_listing() detects a price drop, it calls WatchManager.notify_car_change(), which triggers the update() method on all observers. Each observer evaluates the change against their criteria (e.g., `max_price`) and uses a callback to create a notification if the criteria are met.

2.2.3 Builder Pattern: Car Listing Construction



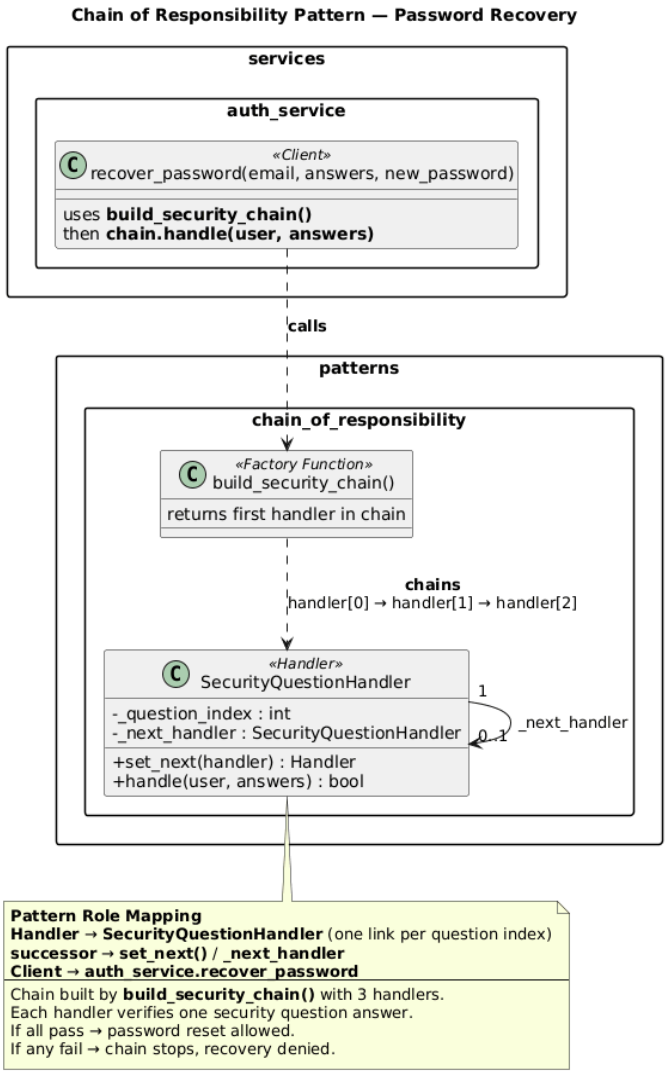

**Description:** The Builder pattern provides a step-by-step, fluent interface for constructing complex car listing objects. This is beneficial because car listings have many optional fields (features, availability windows) and required fields (make, model, year, price, location). The CarListingBuilder accumulates field values through chained setter calls, validates that all required fields are present via validate(), and finally produces the car dictionary via build(). The API route acts as the "Director" that orchestrates the build process based on user input.

2.2.4 Proxy Pattern: Payment Processing



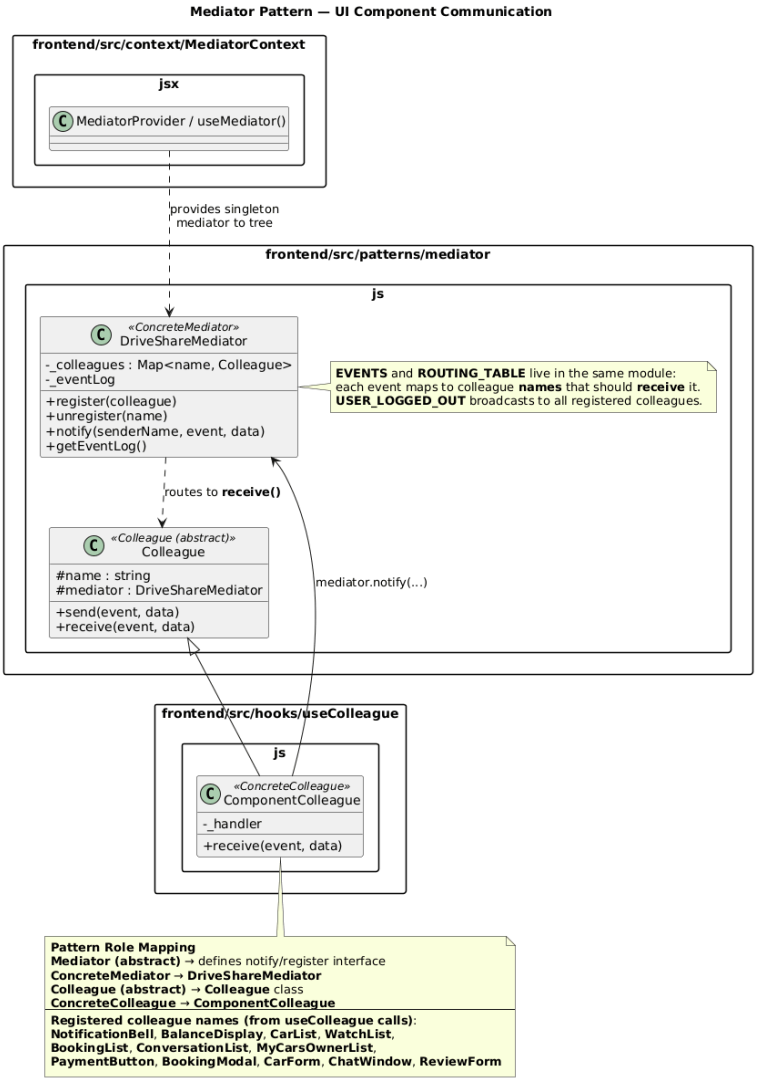

**Description:** The Proxy pattern adds validation, logging, and notification layers around the payment processing logic. The PaymentProxy acts as a middleman: it first validates that the payment amount is positive and that the renter has sufficient balance. If validation passes, it delegates to `RealPaymentService.process()` which performs the actual balance transfers. The proxy then logs the transaction (success or failure), creates notifications for both renter and owner, and marks the associated booking as paid. This separation of concerns keeps the core payment logic simple while adding cross-cutting concerns transparently.

2.2.5 Chain of Responsibility Pattern: Password Recovery




**Description:** The Chain of Responsibility pattern implements the multi-step password recovery process. Each SecurityQuestionHandler is responsible for verifying one of the user's three security question answers. The handlers are linked via set_next() to form a chain. When handle() is called, the first handler checks if the first answer is correct. If so, it passes control to the next handler; if not, it returns False immediately. This continues until all three answers are verified (returns True) or any answer fails (returns False). The factory function build_security_chain() constructs the chain with handlers for indices 0, 1, and 2.

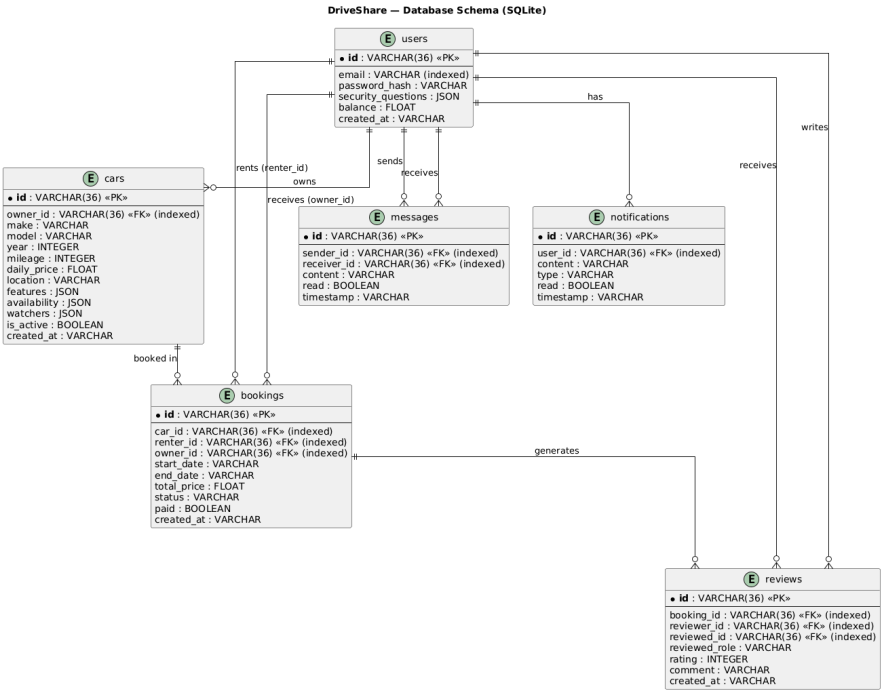
## 2.2.6 Mediator Pattern: UI Component Communication




**Description:** The Mediator pattern decouples React UI components by centralizing their communication through DriveShareMediator. Instead of components directly calling each other, they register as "colleagues" with the mediator and communicate via events. For example, when PaymentButton processes a payment, it sends a BOOKING_PAID event. The mediator routes this to BalanceDisplay (to refresh the balance), BookingList (to update payment status), and NotificationBell (to show a new notification). This pattern eliminates tight coupling between components and makes the event flow explicit and traceable via getEventLog().

# 3. Database Schema

## 3.1 Entity-Relationship Diagram (PlantUML)



3.2 Table Descriptions

**Users**

| Column | Type | Description |
|---|---|---|
| id | VARCHAR(36) | Primary key (UUID) |
| email | VARCHAR | User's email address (indexed for login lookup) |
| password_hash | VARCHAR | SHA-256 hash of the password |
| security_questions | JSON | Array of 3 objects with question and answer fields |
| balance | FLOAT | User's wallet balance (starts at $1000) |
| created_at | VARCHAR | ISO timestamp of account creation |

**Cars**

| Column | Type | Description |
|---|---|---|
| id | VARCHAR(36) | Primary key (UUID) |
| owner_id | VARCHAR(36) | Foreign key to users table |
| make | VARCHAR | Manufacturer (e.g., "Toyota") |
| model | VARCHAR | Model name (e.g., "Camry") |
| year | INTEGER | Model year |
| mileage | INTEGER | Current odometer reading |
| daily_price | FLOAT | Rental price per day |
| location | VARCHAR | Pick-up location |
| features | JSON | Array of feature strings (e.g., ["Bluetooth", "GPS"]) |
| availability | JSON | Array of `{start, end}` date windows |

| | | |
|---|---|---|
| watchers | JSON | Array of user IDs watching this car |
| is_active | BOOLEAN | Soft-delete flag (false = deleted) |
| created_at | VARCHAR | ISO timestamp of listing creation |

**Bookings**

| **Column** | **Type** | **Description** |
|---|---|---|
| id | VARCHAR(36) | Primary key (UUID) |
| car_id | VARCHAR(36) | Foreign key to cars table |
| renter_id | VARCHAR(36) | Foreign key to users (person renting) |
| owner_id | VARCHAR(36) | Foreign key to users (car owner) |
| start_date | VARCHAR | Rental start date (YYYY-MM-DD) |
| end_date | VARCHAR | Rental end date (YYYY-MM-DD) |
| total_price | FLOAT | Calculated total (daily_price × days) |
| status | VARCHAR | "confirmed" or "cancelled" |
| paid | BOOLEAN | Payment completion flag |
| created_at | VARCHAR | ISO timestamp of booking creation |

**Messages**

| Column | Type | Description |
|---|---|---|
| id | VARCHAR(36) | Primary key (UUID) |
| sender_id | VARCHAR(36) | Foreign key to users (message author) |
| receiver_id | VARCHAR(36) | Foreign key to users (recipient) |
| content | VARCHAR | Message text content |
| read | BOOLEAN | Read receipt flag |
| timestamp | VARCHAR | ISO timestamp of when message was sent |

**Notifications**

| Column | Type | Description |
|---|---|---|
| id | VARCHAR(36) | Primary key (UUID) |
| user_id | VARCHAR(36) | Foreign key to users (notification recipient) |
| content | VARCHAR | Notification message text |
| type | VARCHAR | Category: "general", "booking", "payment", "watch", or "message" |
| read | BOOLEAN | Dismissal flag |
| timestamp | VARCHAR | ISO timestamp of account creation |

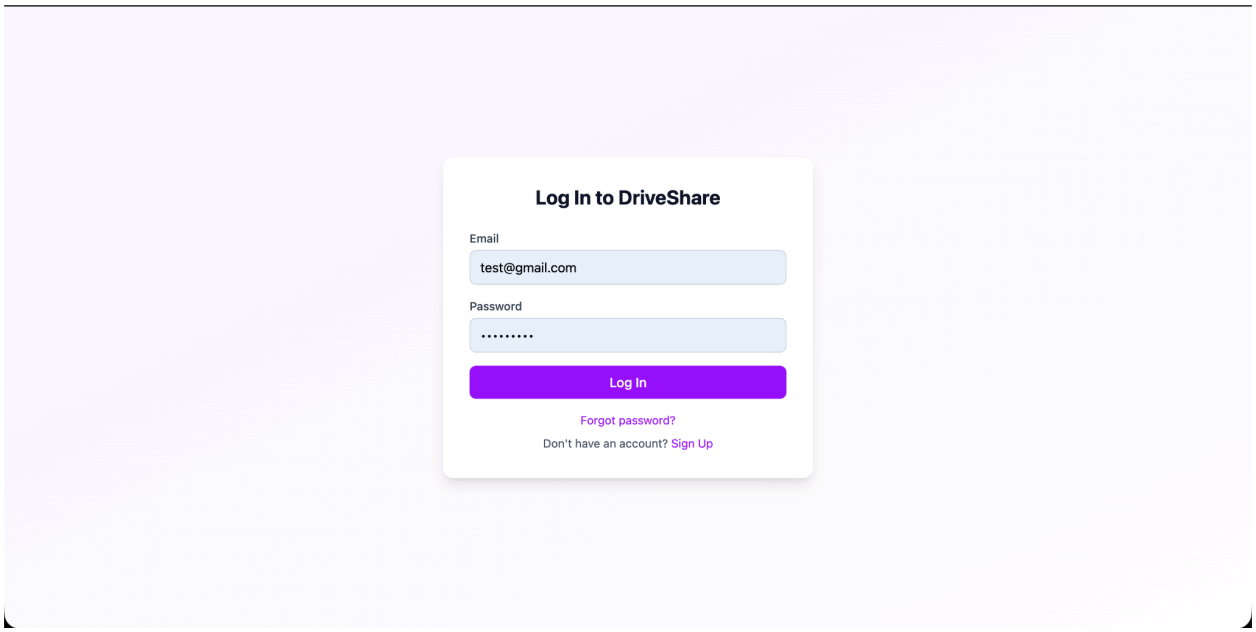
**Reviews**

| Column | Type | Description |
|---|---|---|
| id | VARCHAR(36) | Primary key (UUID) |
| booking_id | VARCHAR(36) | Foreign key to bookings table |
| reviewer_id | VARCHAR(36) | Foreign key to users (person writing review) |
| reviewed_id | VARCHAR(36) | Foreign key to users (person being reviewed) |
| reviewed_role | VARCHAR | "owner" or "renter" |
| rating | INTEGER | Score from 1-5 |
| comment | VARCHAR | Review text |
| created_at | VARCHAR | ISO timestamp of review submission |

4. User Interface Screenshots

### 4.1 Authentication Pages

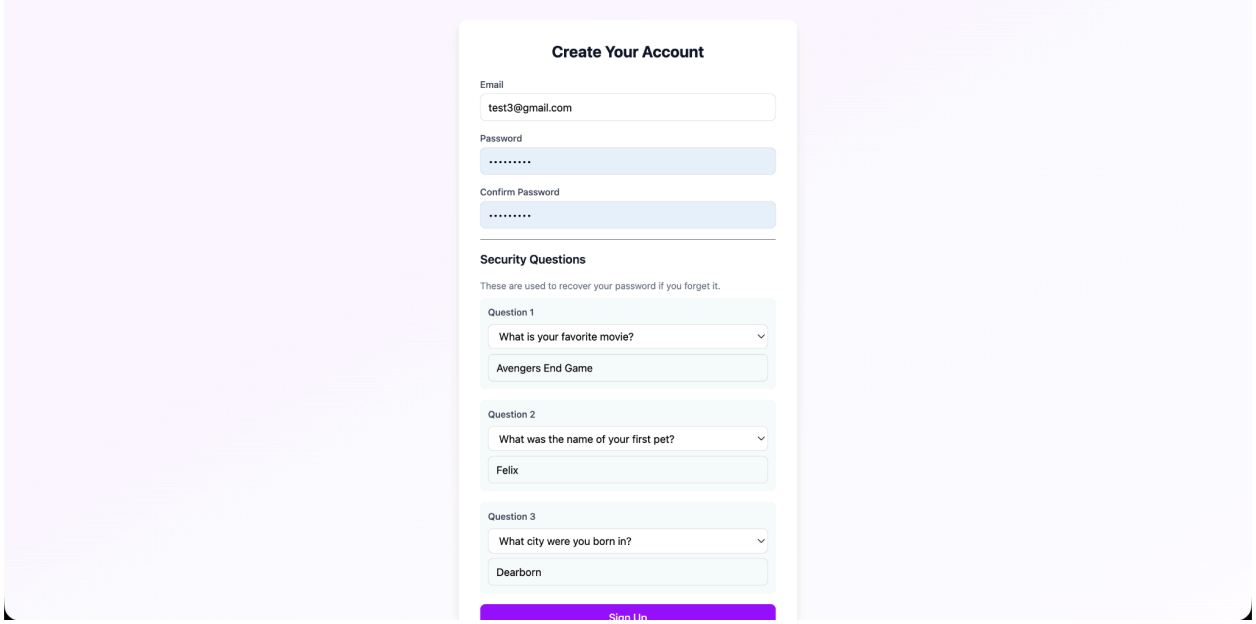
### Login Page




**Description:** A centered login form with a clean, modern design featuring a purple gradient background. The form contains:
- "Log In to DriveShare" header
- Email input field with label
- Password input field with label
- Purple "Log In" button
- "Forgot password?" link below the button
- "Don't have an account? Sign Up" link at the bottom

The form includes an error alert display for invalid credentials and loading state on the submit button.

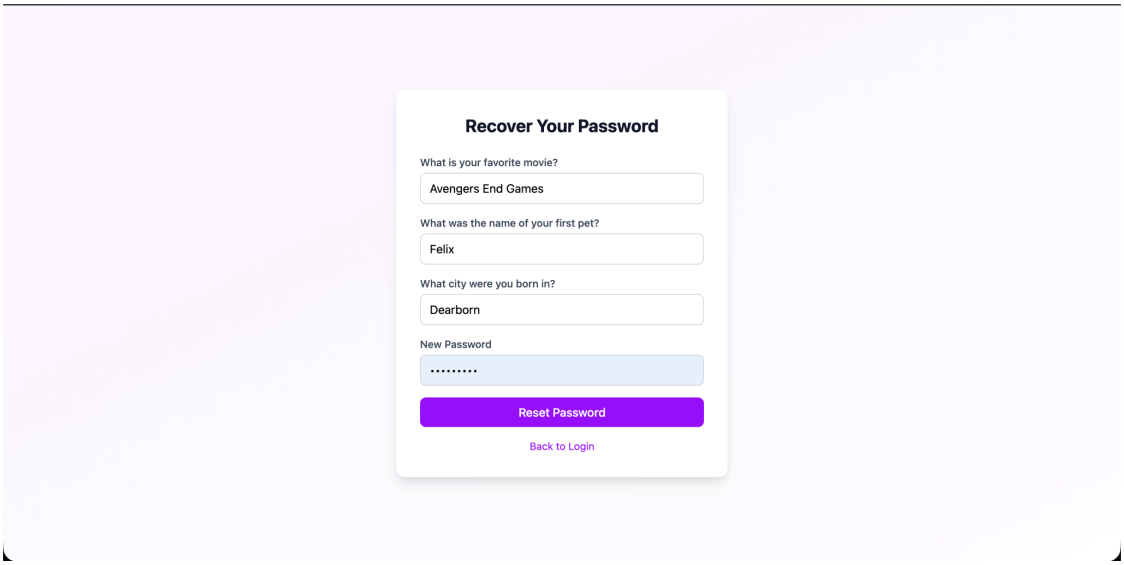
## Registration Page




**Description:** A multi-step registration form with the same purple gradient styling. The form collects:
- Email address
- Password (minimum 4 characters)
- Three security questions with answer fields (for password recovery)
- "Create Account" button

The page displays validation errors inline and redirects to the dashboard upon successful registration.

## Forgot Password Page

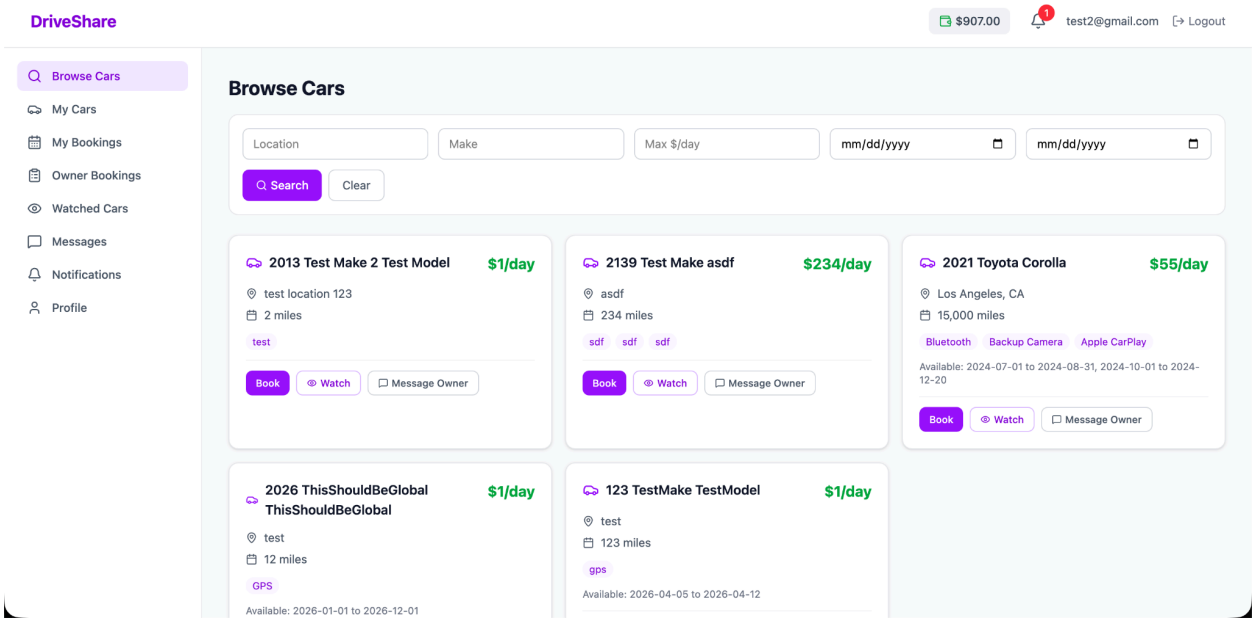



**Description:** A two-step password recovery interface:
**Step 1:** User enters their email address to retrieve their security questions
**Step 2:** User answers all three security questions and enters a new password

The Chain of Responsibility pattern is used behind the scenes to verify each answer sequentially.

## 4.2 Dashboard & Navigation

Dashboard Layout




**Description:** The main authenticated interface with:

- Top Navbar: DriveShare logo, notification bell icon (with unread count badge), user balance display ($X.XX), and logout button
- Left Sidebar: Navigation links including:
    - Browse Cars
    - My Cars
    - My Bookings
    - Owner Bookings
    - Watched Cars
    - Messages
    - Profile
- Main Content Area: Renders the selected page content via React Router outlet

The sidebar collapses on mobile with a hamburger menu toggle.

### 4.3 Car Management

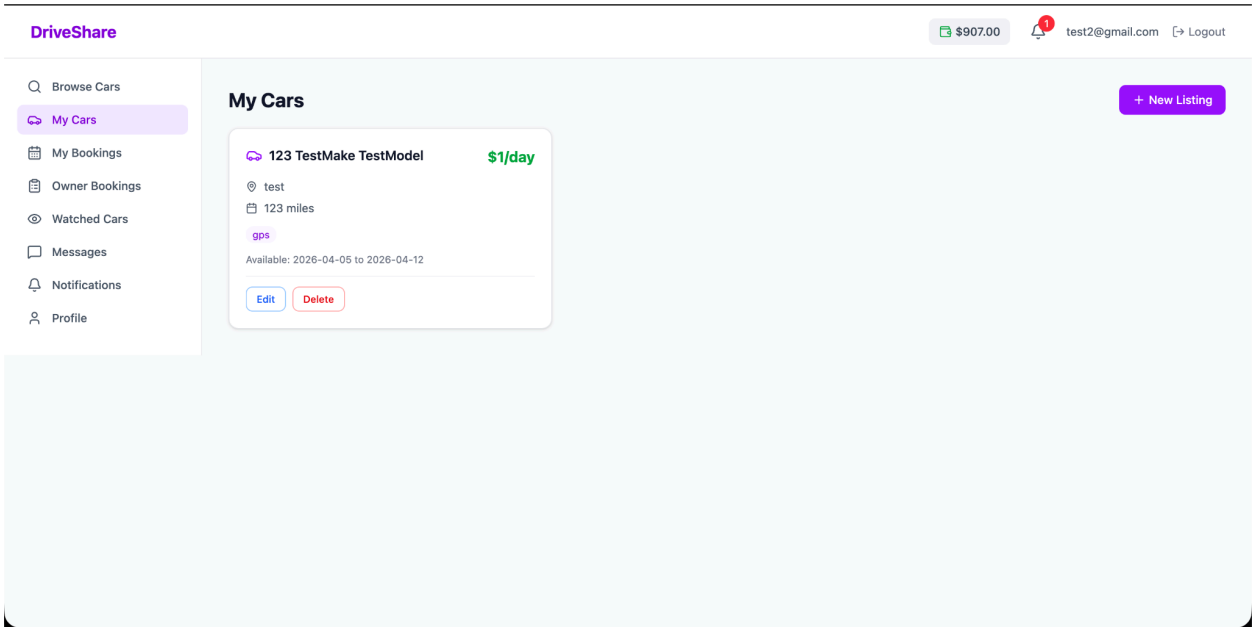
**Browse Cars Page**

**Description:** A searchable grid of available car listings featuring:

- Search Form: Filters for location, date range, max price, and make
- **Car Cards Grid:** Responsive 1/2/3 column layout showing each car with:
    - Make, model, and year
    - Daily price prominently displayed
    - Location
    - Feature tags (Bluetooth, GPS, etc.)
    - "Book" button to open booking modal
    - "Watch" button to monitor price drops
    - "Message Owner" button

The page uses the Mediator pattern to refresh when cars are added, updated, or deleted.

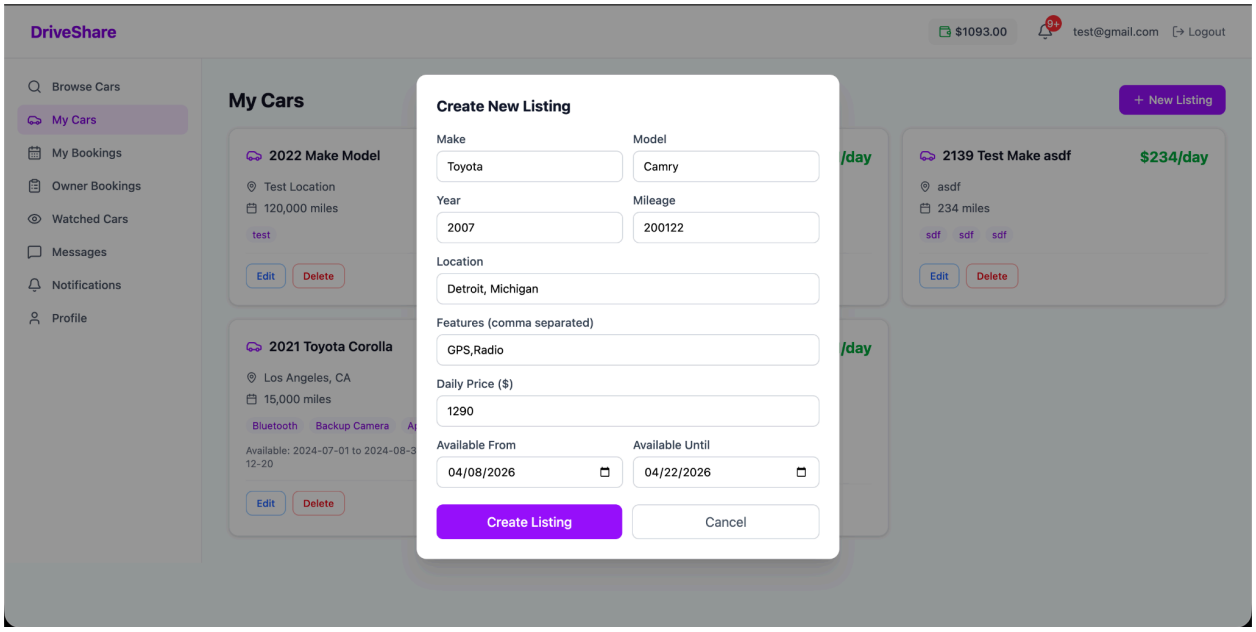
**My Cars Page (Owner View)**




**Description:** Owner's dashboard for managing their car listings with:

- "My Cars" header with "New Listing" button (purple, with plus icon)
- Grid of owned car cards, each showing:
    - Car details (make, model, year, price, location)
    - "Edit" button to modify listing
    - "Delete" button to remove listing
- Empty state message: "You haven't listed any cars yet."

## Create/Edit Car Listing Form

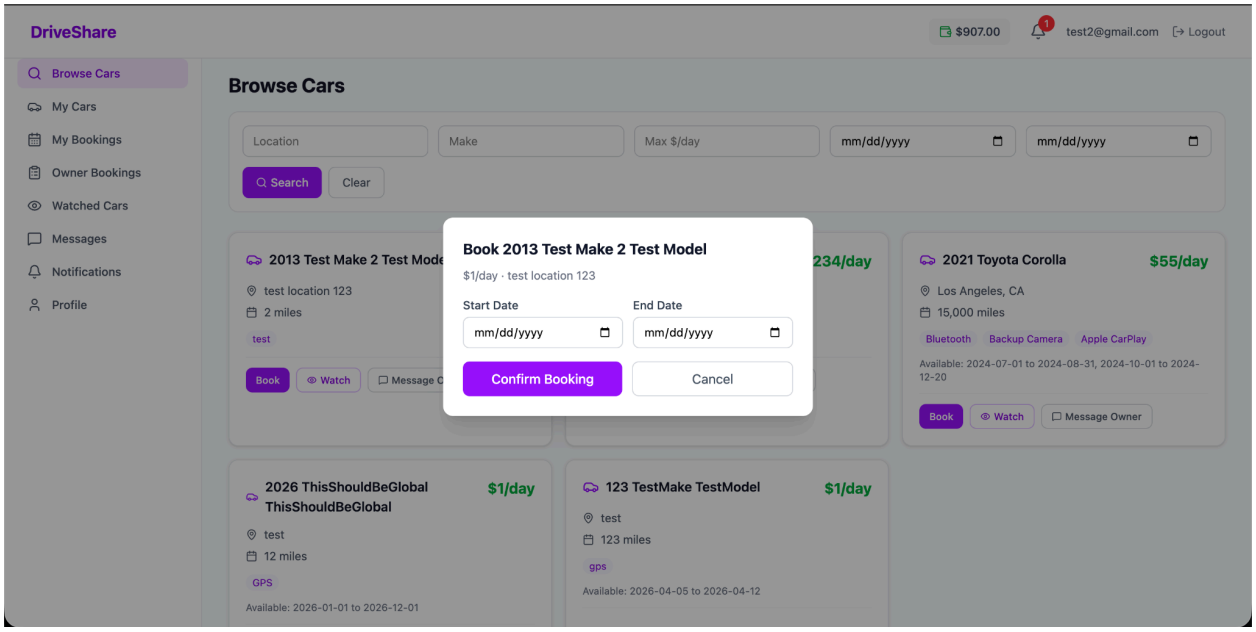



**Description:** A modal form for creating or editing car listings with:
- Make input field
- Model input field
- Year input field
- Mileage input field
- Daily price input field
- Location input field
- Features input (comma-separated)
- Availability date ranges (add/remove windows)
- "Save" and "Cancel" buttons

The Builder pattern is used on the backend to construct the car object from form data.

## 4.4 Booking System

### Book a Car Modal

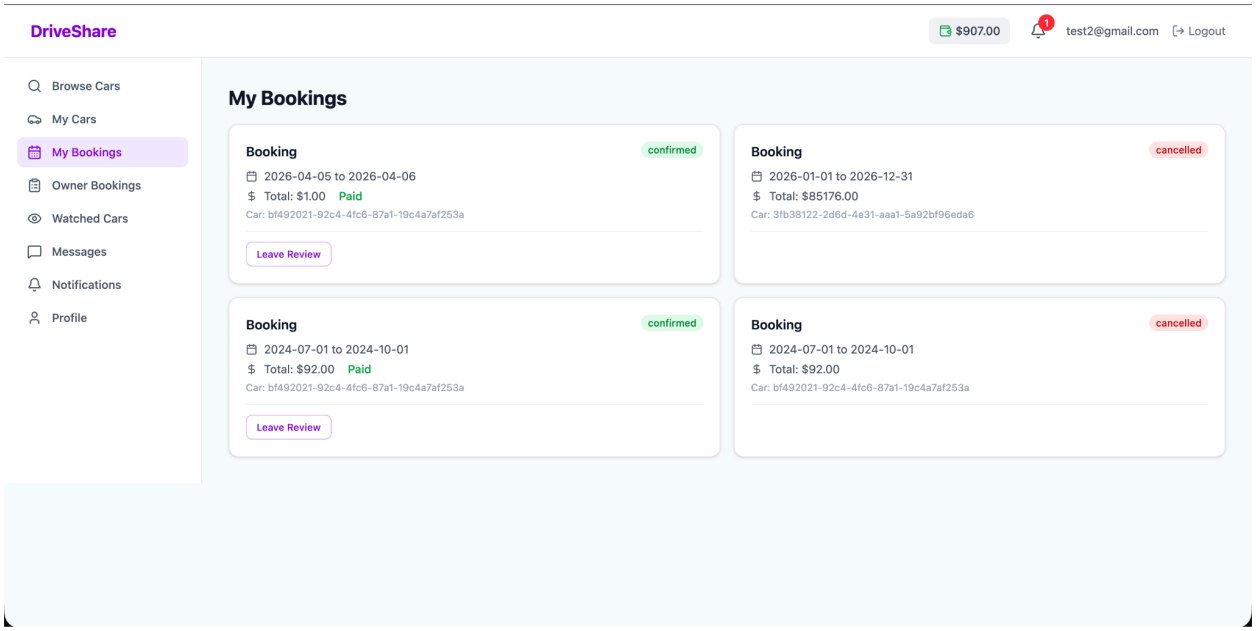



**Description:** A modal dialog for creating a new booking:

- Car summary at top (make, model, daily price)
- Start date picker
- End date picker
- Calculated total price display (days × daily_price)
- "Confirm Booking" button
- "Cancel" button

Includes validation for date overlap and prevents booking own cars.

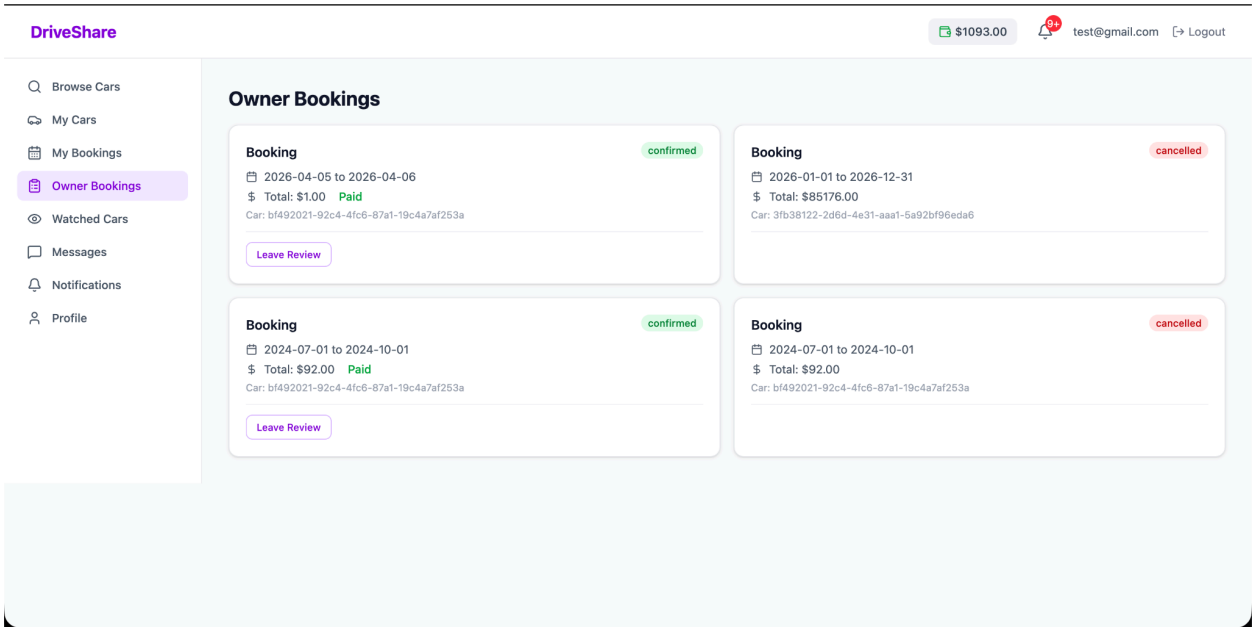
## My Bookings Page (Renter View)




**Description:** List of bookings where the user is the renter:

- Booking cards showing:
  - Car info (make, model)
  - Date range (start – end)
  - Total price
  - Status badge (Confirmed/Cancelled)
  - Payment status (Paid/Unpaid)
  - "Pay Now" button (if unpaid) (uses Proxy pattern)
  - "Cancel Booking" button
  - "Leave Review" button (after rental complete)

## Owner Bookings Page

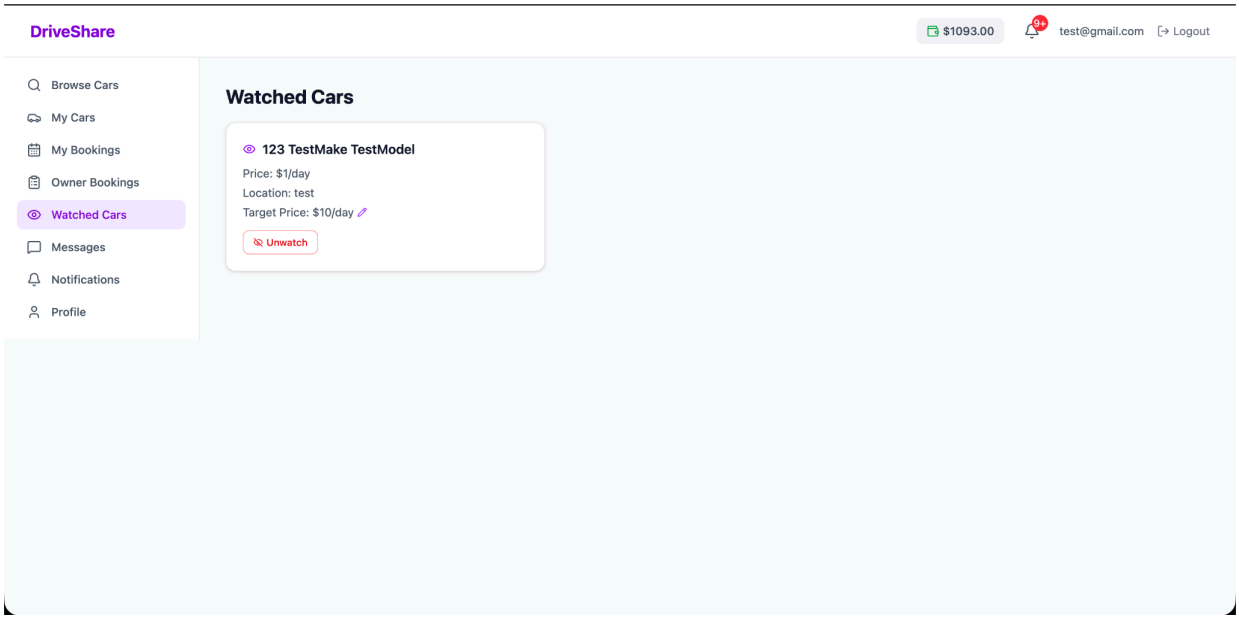



**Description:** List of bookings for cars the user owns:

- Similar card layout to My Bookings
- Shows renter's email
- "Message Renter" button for communication
- Payment received status

## 4.5 Watch & Notifications

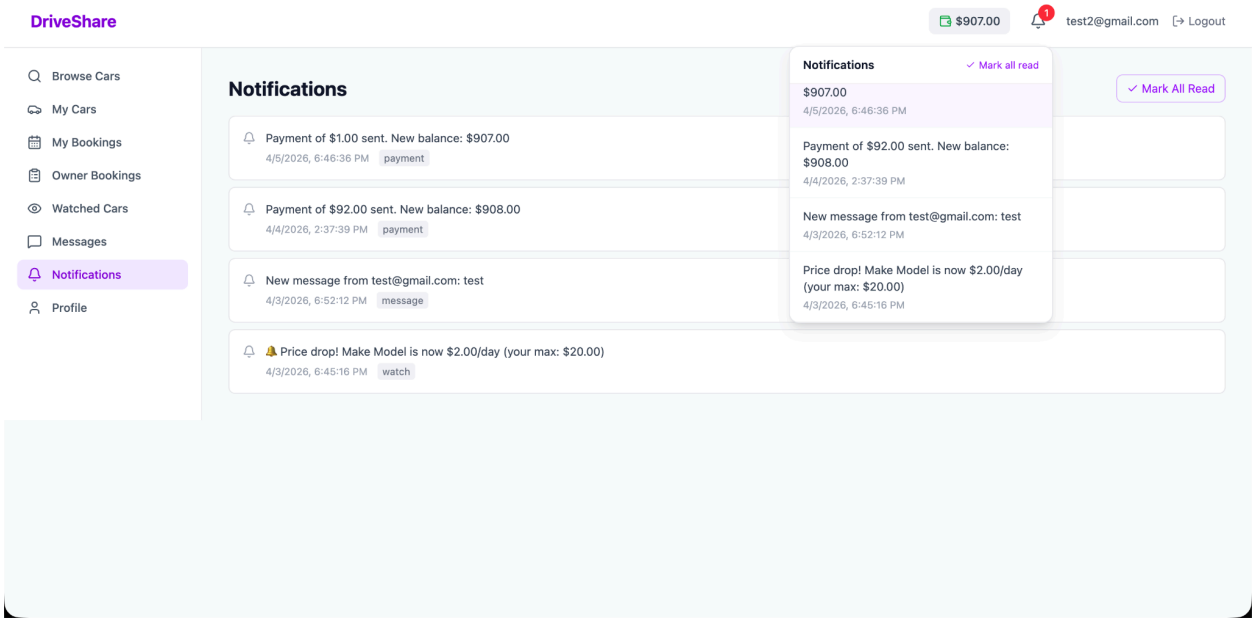
### Watched Cars Page




**Description:** List of cars the user is watching for price changes:

- Car cards with current price
- Target price set by user
- "Unwatch" button to remove from watch list
- Notification appears when price drops below target (Observer pattern)

### Notifications Panel

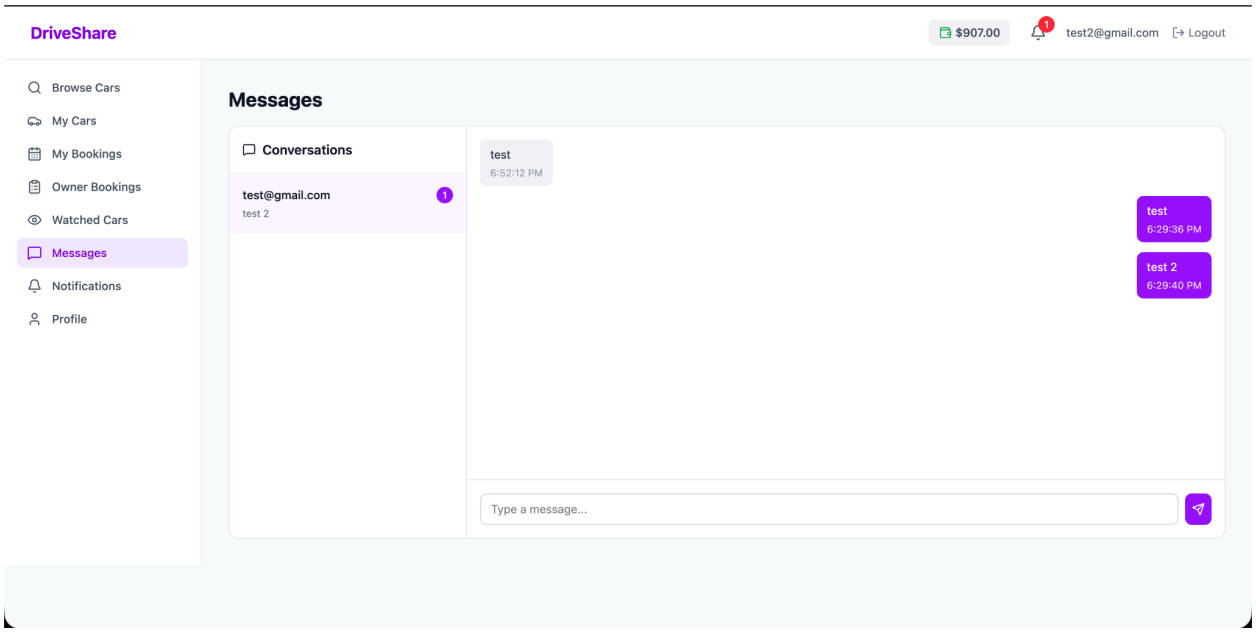


**Description:** Dropdown panel from the notification bell showing:
List of notifications with icons by type:

- Booking notifications
    - Payment notifications
    - Watch/price drop notifications
    - Message notifications
- Timestamp for each notification
    - "Mark as Read" action
    - Unread count badge on bell icon

## 4.6 Messaging

### Messages Page

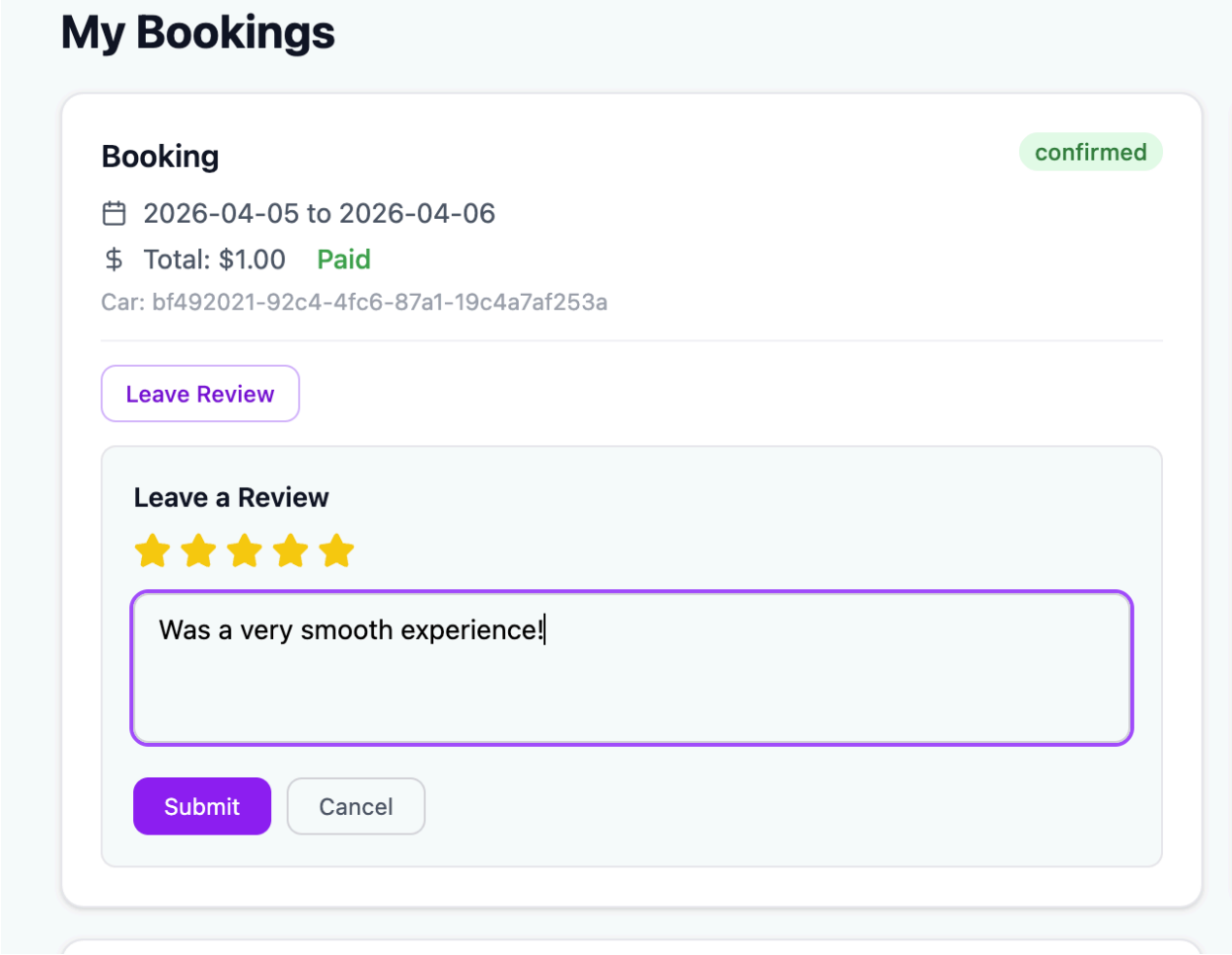



**Description:** Split-pane messaging interface:

- Left Panel: Conversation list showing other users with unread message count
- Right Panel: Chat window displaying: Message bubbles (sent messages on right, received on left), Timestamps, Message input field at bottom, and "Send" button

4.7 Reviews & Profile

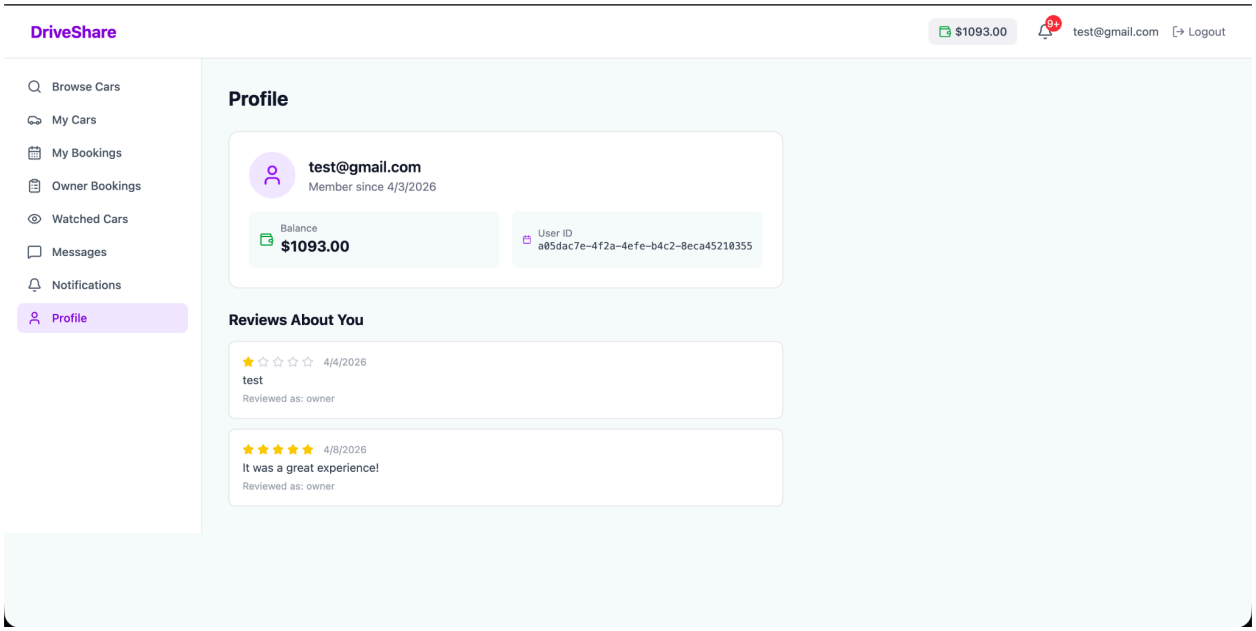
**Review Form**




**Description:** Post-rental review submission form:

- Star rating selector (1-5 stars)
- Comment text area
- "Submit Review" button
- Appears after booking is complete and paid

## Profile Page




**Description:** User profile page showing:

- Email address
- Current wallet balance
- Reviews received (as owner and as renter)
- Average rating display

5. References

5.1 Design Patterns


1. **Refactoring Guru - Design Patterns**
    - https://refactoring.guru/design-patterns/singleton
    - https://refactoring.guru/design-patterns/observer
    - https://refactoring.guru/design-patterns/builder
    - https://refactoring.guru/design-patterns/proxy
    - https://refactoring.guru/design-patterns/chain-of-responsibility
    - https://refactoring.guru/design-patterns/mediator
2. **CIS 476 Lectures / Lecture Notes**
    - https://canvas.umd.umich.edu/courses/547926/files/folder/Lecture%20Notes


**5.2 Technologies & Frameworks**


1. **Python Documentation** - https://docs.python.org/3/
    - hashlib (SHA-256 hashing): https://docs.python.org/3/library/hashlib.html
    - uuid: https://docs.python.org/3/library/uuid.html

2. **FastAPI Documentation** - https://fastapi.tiangolo.com/
    - API routing and request handling

3. **SQLAlchemy Documentation** - https://docs.sqlalchemy.org/
    - ORM for SQLite database operations

4. **React Documentation** - https://react.dev/
    - Component-based UI development
    - Context API for state management
    - Hooks (useState, useEffect, useCallback)

5. **React Router** - https://reactrouter.com/
    - Client-side routing and navigation